Lab 3: Component Modelling & Architectural Pattern Selection

Self-Service Coffee Kiosk System — Written Justification

Student SRN:	PES1UG24CS537
Course:	Software Engineering Lab
Lab:	Lab 3 — Architecture Selection & Component Diagram
Date:	October 5, 2024

---

1. Architecture Selection

We chose Client-Server Architecture for the Self-Service Coffee Kiosk System.

2. Reasons for Architectural Choice

Reason 1 — Centralised Control over Menu and Pricing:

The kiosk must display accurate coffee types (Espresso, Americano, Latte) with correct pricing at all times. A Client-Server model places all menu and pricing data on a centralised server component (*Menu & Pricing DB*), ensuring every transaction reads from a single authoritative source. This eliminates data inconsistencies that would arise if each kiosk stored its own local copy, and makes menu updates (e.g., price changes, seasonal items) instant without touching the kiosk software itself.

Reason 2 — Simplified Deployment and Maintenance for a Single-Location Kiosk:

The scenario describes a busy café with a single self-service kiosk — a scope that does not demand the operational overhead of a Microservices architecture. Client-Server architecture is inherently straightforward to deploy: the server components (Order Manager, Payment Service, Menu & Pricing DB, Receipt Printer Service) run on a back-end host, while the Touch Screen UI acts as the thin client. This dramatically lowers infrastructure and maintenance costs compared to running independent microservice containers, and is far easier to manage for café staff without dedicated DevOps support.

3. Security Advantage

The Client-Server model confines all payment processing to the server-side *Payment Service* component, which communicates with the card terminal over an encrypted, internal channel (IPaymentProcess interface). The Touch Screen UI — the most exposed component, since it handles physical customer interactions — never directly touches raw card data. This isolation enforces the principle of least privilege: the UI component can only submit a payment request; it cannot read, store, or forward card credentials. Sensitive data therefore remains protected on the server side, reducing the attack surface for physical tampering or UI-layer exploits.

4. Performance Benefit

Because the Order Manager and associated server components run on dedicated hardware separate from the touch-screen client, compute-intensive tasks — credit card authorisation, receipt generation, and database queries — do not compete with UI rendering resources. The touch screen remains responsive to customer input (selections, confirmations) even while the back end processes the payment, queries pricing, and spools the receipt in parallel. This separation of client and server workloads ensures a smooth, lag-free ordering experience, which is critical for throughput in a busy café environment.